

SQL LAB-21

Implement Intermediate Common Table Expressions (CTE) for Query Simplification

From the table STUDENT perform the following queries:

Part – A

1. **Display all students whose SPI is greater than 8.**

```
WITH CTE AS (  
    SELECT * FROM STUDENT  
)  
SELECT *  
FROM CTE  
WHERE SPI > 8
```

2. **Display average SPI of all students.**

```
WITH CTE AS (  
    SELECT SPI FROM STUDENT  
)  
SELECT AVG(SPI) AS AVG_SPI  
FROM CTE
```

3. **Display total number of students in each branch.**

```
WITH CTE AS (  
    SELECT BRANCH, COUNT(*) AS TOTAL_STUDENTS  
    FROM STUDENT  
    GROUP BY BRANCH  
)  
SELECT * FROM CTE
```

4. Display students who belong to RAJKOT city.

```
WITH CTE AS (  
    SELECT * FROM STUDENT  
)  
SELECT *  
FROM CTE  
WHERE CITY = 'RAJKOT'
```

5. Find branch names that appear more than once.

```
WITH CTE AS (  
    SELECT BRANCH, COUNT(*) AS CNT  
    FROM STUDENT  
    GROUP BY BRANCH  
)  
SELECT BRANCH  
FROM CTE  
WHERE CNT > 1
```

6. Display row number for each student.

```
WITH CTE AS (  
    SELECT *,  
        ROW_NUMBER() OVER (ORDER BY STDID) AS RN  
    FROM STUDENT  
)  
SELECT *  
FROM CTE
```

7. Display top 3 students based on SPI.

```
WITH CTE AS (  
    SELECT *,  
           ROW_NUMBER() OVER (ORDER BY SPI DESC) AS RN  
    FROM STUDENT  
)  
SELECT *  
FROM CTE  
WHERE RN <= 3
```

8. Display students having maximum SPI.

```
WITH CTE AS (  
    SELECT MAX(SPI) AS MAX_SPI  
    FROM STUDENT  
)  
SELECT S.*  
FROM STUDENT S  
JOIN CTE C  
ON S.SPI = C.MAX_SPI
```

9. Display students having minimum SPI.

```
WITH CTE AS (  
    SELECT MIN(SPI) AS MIN_SPI  
    FROM STUDENT  
)  
SELECT S.*  
FROM STUDENT S  
JOIN CTE C  
ON S.SPI = C.MIN_SPI
```

10. Display branch-wise rank of students.

```
WITH CTE AS (  
    SELECT *,  
        RANK() OVER (  
            PARTITION BY BRANCH  
            ORDER BY SPI DESC  
        ) AS BRANCH_RANK  
    FROM STUDENT  
)  
SELECT *  
FROM CTE
```

Part – B

11. Display students SPI average belonging to Computer branch.

```
WITH CTE AS (  
    SELECT SPI  
    FROM STUDENT  
    WHERE BRANCH = 'COMPUTER'  
)  
SELECT AVG(SPI) AS AVG_SPI  
FROM CTE
```

12. Display students whose SPI is greater than average SPI of his/her branch.

```
WITH CTE AS (  
    SELECT *,  
           AVG(SPI) OVER (PARTITION BY BRANCH) AS  
           BRANCH_AVG  
    FROM STUDENT  
)  
SELECT *  
FROM CTE  
WHERE SPI > BRANCH_AVG
```

13. Display branch having more than 2 students.

```
WITH CTE AS (  
    SELECT BRANCH, COUNT(*) AS TOTAL  
    FROM STUDENT  
    GROUP BY BRANCH  
)  
SELECT BRANCH, TOTAL  
FROM CTE  
WHERE TOTAL > 2
```

14. Display branches having average SPI between 7 and 9.

```
WITH CTE AS (  
    SELECT BRANCH, AVG(SPI) AS AVG_SPI  
    FROM STUDENT  
    GROUP BY BRANCH  
)  
SELECT *  
FROM CTE  
WHERE AVG_SPI BETWEEN 7 AND 9
```

15. Display students whose SPI is lower than overall average SPI.

```
WITH CTE AS (  
    SELECT AVG(SPI) AS AVG_SPI  
    FROM STUDENT  
)  
SELECT *  
FROM STUDENT  
WHERE SPI < (SELECT AVG_SPI FROM CTE);
```

OR

```
WITH CTE AS (  
    SELECT AVG(SPI) AS AVG_SPI  
    FROM STUDENT  
)  
SELECT S.*  
FROM STUDENT S, CTE C  
WHERE S.SPI < C.AVG_SPI;
```

OR

```
WITH CTE AS (  
    SELECT *,  
        AVG(SPI) OVER () AS OVERALL_AVG  
    FROM STUDENT  
)  
SELECT *  
FROM CTE  
WHERE SPI < OVERALL_AVG;
```

Part – C

16. Display branches having exactly one student.

```
WITH CTE AS (  
    SELECT BRANCH, COUNT(*) AS TOTAL  
    FROM STUDENT  
    GROUP BY BRANCH  
)  
SELECT BRANCH  
FROM CTE  
WHERE TOTAL = 1
```

17. Display branch having highest average SPI.

```
WITH CTE1 AS (  
    SELECT BRANCH, AVG(SPI) AS AVG_SPI  
    FROM STUDENT  
    GROUP BY BRANCH  
) ,  
CTE2 AS (  
    SELECT MAX(AVG_SPI) AS MAX_AVG  
    FROM CTE  
)  
SELECT CTE1.*  
FROM CTE1  
JOIN CTE2  
ON CTE1.AVG_SPI = CTE2.MAX_AVG
```

18. Display branch having lowest average SPI.

```
WITH CTE1 AS (  
    SELECT BRANCH, AVG(SPI) AS AVG_SPI  
    FROM STUDENT  
    GROUP BY BRANCH  
)  
CTE2 AS (  
    SELECT MIN(AVG_SPI) AS MIN_AVG  
    FROM CTE  
)  
SELECT CTE1.*  
FROM CTE1  
JOIN CTE2  
ON CTE1.AVG_SPI = CTE2.MIN_AVG
```

19. Display students whose SPI is lower than branch average SPI.

```
WITH CTE AS (  
    SELECT *,  
           AVG(SPI) OVER (PARTITION BY BRANCH) AS  
           BRANCH_AVG  
    FROM STUDENT  
)  
SELECT *  
FROM CTE  
WHERE SPI < BRANCH_AVG
```


20. Display branches having maximum number of students.

```
WITH CTE1 AS (  
    SELECT BRANCH, COUNT(*) AS TOTAL  
    FROM STUDENT  
    GROUP BY BRANCH  
)  
CTE2 AS (  
    SELECT MAX(TOTAL) AS MAX_TOTAL  
    FROM CTE  
)  
SELECT CTE1.*  
FROM CTE1  
JOIN CTE2  
ON CTE1.TOTAL = CTE2.MAX_TOTAL;
```